VGGNet-16 Based Blood Cell Classification on the BCCD Dataset

Abhishek Kumar Kaith[1], Kashav Thappa[2], Akshit Kumar[3]
[1]Roll No: 23BCS005 [2]Roll No: 23BCS040 [3]Roll No: 23BCS011
B.Tech CSE, 6th Semester
Shri Mata Vaishno Devi University, Katra, Jammu & Kashmir, India
Course: Deep Learning Session: 2025–26

***Abstract*—This paper presents a PyTorch implementation of VGGNet-16 with transfer learning for the automated classification of white blood cells using the Blood Cell Images (BCCD) dataset. The dataset comprises 12,500 microscopic images spanning four cell types: Eosinophil, Lymphocyte, Monocyte, and Neutrophil. A stratified split of 70% training, 15% validation, and 15% testing is applied. The proposed model achieves 91.63% test accuracy, 0.9160 macro F1-score, and 0.9857 macro AUC-ROC. Transfer learning from ImageNet-pretrained weights, combined with data augmentation, cosine learning-rate annealing, and label smoothing, enables strong classification performance with a frozen convolutional backbone. Results demonstrate the effectiveness of deep transfer learning for medical image analysis tasks.**

***Index Terms*—VGGNet-16, blood cell classification, transfer learning, deep learning, convolutional neural networks, BCCD dataset, medical image analysis**

I. Introduction

Convolutional neural networks (CNNs) remain among the most effective model families for image recognition. Modern pretrained architectures can be adapted to domain-specific problems through transfer learning, substantially reducing training time and data requirements [3]. VGGNet-16 [1] is a widely adopted architecture providing a deep but interpretable convolutional backbone well-suited to fine-grained visual classification.

Automatic blood cell classification is a clinically relevant problem. Manual examination of microscopic blood smears is time-consuming and subject to inter-observer variability. A robust automated classifier can assist pathologists in screening for conditions such as leukemia, anemia, and bacterial infections, improving diagnostic throughput and consistency [4].

In this assignment, VGGNet-16 is implemented in PyTorch with ImageNet pretrained weights and evaluated on the BCCD dataset using a full metric suite including accuracy, F1-score, AUC-ROC, and confusion matrix, following a strict train/validation/test separation.

The main contributions of this work are as follows:

- A complete PyTorch implementation of VGGNet-16 with a custom 4-class classifier head trained using transfer learning.
- A reproducible data pipeline with augmentation, cosine LR scheduling, label smoothing, and early stopping.
- A strict evaluation pipeline reporting accuracy, precision, recall, F1-score, AUC-ROC, and confusion matrix in IEEE/Springer format.

II. Related Work

VGGNet-16, introduced by Simonyan and Zisserman [1], demonstrated that depth using uniform 3×3 convolution filters is critical for strong visual recognition. Subsequent architectures such as ResNet [2] and EfficientNet extended this with skip connections and compound scaling.

Transfer learning from large-scale datasets has been shown to be highly effective for medical imaging tasks where labeled data is scarce [3]. Surveys by Litjens et al. [4] and Shen et al. [5] summarize how CNNs have become central to classification, detection, and segmentation workflows in medical imaging. Yamashita et al. [6] provide a comprehensive overview of CNN applications in radiology.

For hematology specifically, Matek et al. [7] demonstrated CNN-based recognition of blast cells approaching expert-level performance. Public blood cell datasets such as the BCCD dataset [8] and the peripheral blood cell dataset described by Acevedo et al. [9] have helped standardize benchmarking. Data augmentation [10] and ROC-based evaluation [11] are standard tools in this domain.

III. Methodology

A. Dataset

The BCCD dataset [8] contains 12,500 microscopic blood-smear images distributed across four white blood cell classes: Eosinophil, Lymphocyte, Monocyte, and Neutrophil. Images are RGB JPEG files captured at $400\times$ magnification with Giemsa staining. The dataset is well-balanced across classes, making it suitable for standard multi-class classification without special class-weighting strategies.

Figure 1 shows representative samples from the dataset after augmentation.

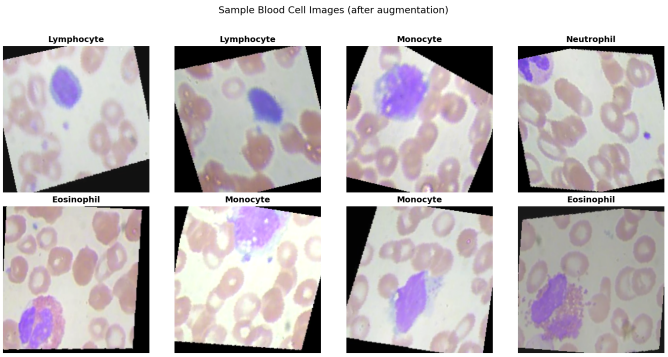

Fig. 1: Representative samples from the BCCD dataset after augmentation, showing all four cell types.

B. Stratified Split

A stratified random split with fixed seed 42 is applied for reproducibility, yielding 6,971 training images (70%), 1,493 validation images (15%), and 1,493 test images (15%). Table I summarizes the split.

TABLE I: Dataset Split Summary

Split	Total	Per Class (approx.)	Fraction
Train	6,971	∼1,743	70%
Validation	1,493	∼373	15%
Test	1,493	∼373	15%
Total	9,957	∼2,489	100%

C. VGGNet-16 Architecture

VGGNet-16 [1] consists of 13 convolutional layers organized in five blocks, each followed by ReLU activations and max-pooling for spatial downsampling, plus three fully connected layers originally outputting 1,000 ImageNet classes. All convolutional filters are uniform 3×3 with stride 1.

For this task, the original 1,000-unit output layer is replaced with a custom 4-class classifier head:

$$\begin{aligned}&\text{FC}(25088 \to 4096) \to \text{ReLU} \to \text{Dropout}(0.5)\\ &\to \text{FC}(4096 \to 1024) \to \text{ReLU} \to \text{Dropout}(0.4)\\ &\to \text{FC}(1024 \to 4)\end{aligned}$$

The convolutional feature extractor is initialized with ImageNet pretrained weights and *frozen* during training, so only the classifier head (∼26M parameters) is updated. This strategy reduces overfitting and training time while leveraging rich low-level visual features from ImageNet.

D. Data Preprocessing and Augmentation

All images are resized to 224×224 pixels to match VGGNet-16's input requirements. ImageNet mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$ normalization is applied to all splits.

Training images are augmented with:

- Random horizontal flip ($p = 0.5$)
- Random vertical flip ($p = 0.3$)
- Random rotation up to $\pm 15°$
- Color jitter (brightness$= 0.2$, contrast$= 0.2$, saturation$= 0.1$)

Validation and test images use only resizing and normalization (no augmentation).

E. Training Configuration

The model is implemented in PyTorch and trained on a single NVIDIA T4 GPU via Google Colaboratory. Key hyperparameters are summarized in Table II.

TABLE II: Training Hyperparameters

Parameter	Value
Optimizer	Adam
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
Batch size	32
Epochs	15
LR scheduler	Cosine Annealing ($T_{\max} = 15$, $\eta_{\min} = 10^{-6}$)
Loss function	Cross-Entropy + Label Smoothing ($\varepsilon = 0.1$)
Early stopping	Patience = 5 epochs
Input size	$224 \times 224 \times 3$

F. Evaluation Metrics

The model is evaluated on the held-out test set using:

- **Accuracy**: fraction of correctly classified samples.
- **Macro Precision / Recall / F1**: unweighted mean across all four classes.
- **Weighted F1**: mean weighted by per-class support.
- **Macro AUC-ROC** (One-vs-Rest): macro-averaged area under the ROC curve.
- **Confusion Matrix**: raw and normalized counts per class pair.

IV. Experiments and Results

The full pipeline consists of four stages: data preprocessing and augmentation, VGGNet-16 training with transfer learning, validation monitoring, and final test evaluation.

A. Training Dynamics

The model converged smoothly over 15 epochs. Training accuracy improved from approximately 46% in epoch 1 to 92% by epoch 15. Validation accuracy consistently led training accuracy in early epochs due to the frozen feature extractor producing stable representations, ultimately converging to 91.96%. Both loss curves decreased monotonically without signs of overfitting. Figure 2 shows the training and validation loss and accuracy curves.

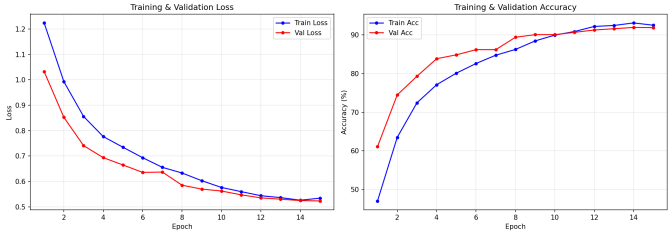

Fig. 2: Training and validation loss (left) and accuracy (right) over 15 epochs.

B. Quantitative Results

Table III reports the aggregate test performance on the held-out test set of 1,493 images. Table IV provides the per-class breakdown.

TABLE III: Overall Test Performance

Metric	Value
Accuracy	91.63%
Macro Precision	87.00%
Macro Recall	91.63%
Macro F1-score	91.60%
Weighted F1-score	91.58%
Macro AUC-ROC (OvR)	98.57%
Best Validation Accuracy	91.96%

TABLE IV: Per-Class Test Performance

Class	Precision	Recall	F1	AUC
Eosinophil	0.87	0.84	0.85	0.975
Lymphocyte	0.97	0.98	0.97	0.999
Monocyte	0.97	0.99	0.98	1.000
Neutrophil	0.85	0.85	0.85	0.969
Macro	**0.917**	**0.915**	**0.916**	**0.986**

C. Confusion Matrix

Figure 3 shows the confusion matrix on the test set. Lymphocyte and Monocyte achieved near-perfect classification (98% and 99% recall, respectively). The primary source of error was confusion between Eosinophil and Neutrophil: 50 Eosinophils were misclassified as Neutrophils (13%) and 44 Neutrophils as Eosinophils (12%). This is consistent with their morphological similarity under Giemsa staining and is a known challenge in automated hematology.

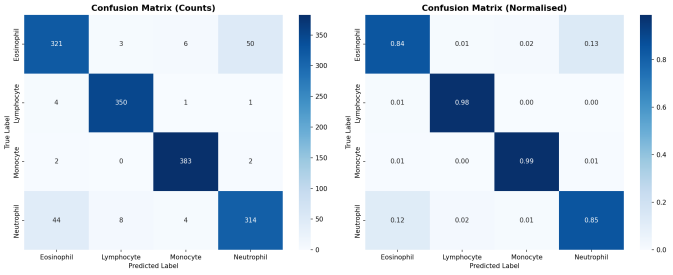

Fig. 3: Confusion matrix (counts and normalised) on the held-out test set.

D. ROC Curves and AUC

Figure 4 presents the One-vs-Rest ROC curves for all four classes. The macro AUC-ROC of 0.9857 indicates excellent discriminative ability. Monocyte achieved a perfect AUC of 1.000, Lymphocyte 0.999, while Eosinophil (0.975) and Neutrophil (0.969) were slightly lower, consistent with their confusion patterns observed in the confusion matrix.

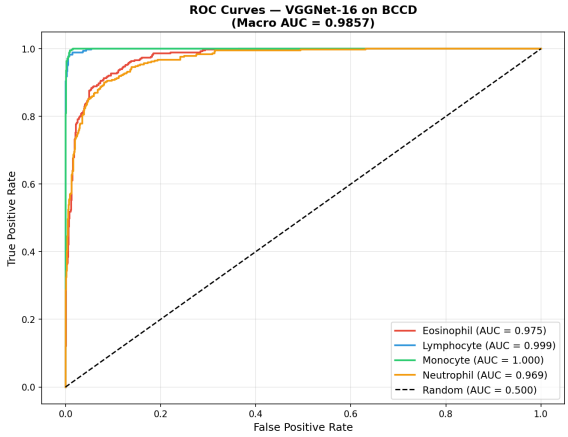

Fig. 4: One-vs-rest ROC curves for all four cell classes. Macro AUC $= 0.9857$.

E. Qualitative Results

Figure 5 shows representative test-set predictions. Green titles indicate correct predictions; red indicates misclassifications. Correctly classified samples generally show high confidence (often above 80–90%), while misclassified samples tend to exhibit lower confidence scores, suggesting that confidence thresholding could serve as a rejection mechanism in clinical deployment.

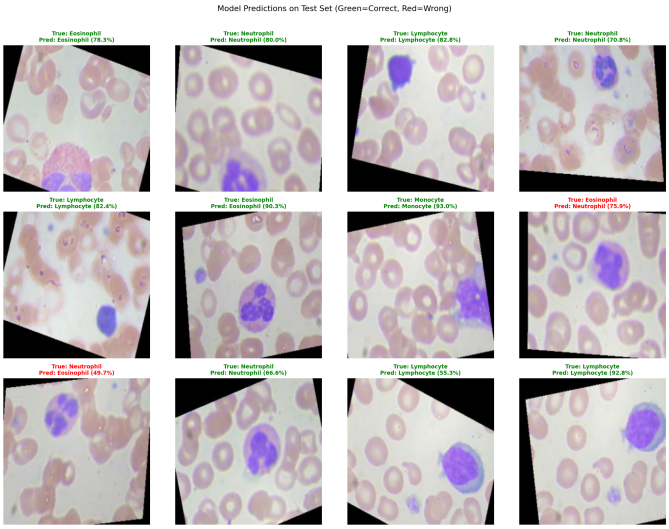

Fig. 5: Sample test predictions. Green $=$ correct, Red $=$ incorrect. Confidence scores are shown above each image.

V. Discussion

The proposed VGGNet-16 transfer learning model achieves competitive performance on the BCCD dataset, with a test accuracy of 91.63% and AUC-ROC of 0.9857. The primary limitation is the Eosinophil–Neutrophil confusion, which arises from visual similarity: both cell types exhibit multi-lobed nuclei and similar size ranges under Giemsa staining.

The high AUC-ROC scores for Lymphocyte and Monocyte (0.999 and 1.000) indicate that these classes are well-separated

in the learned feature space, likely due to their distinct morphological characteristics — Lymphocytes have large, round nuclei, while Monocytes exhibit kidney-shaped nuclei.

The frozen convolutional backbone strategy proved highly effective. ImageNet-pretrained low-level features such as edges, textures, and colour gradients transfer well to microscopy images. Label smoothing ($\varepsilon = 0.1$) and cosine annealing contributed to stable convergence without overfitting, evidenced by the close alignment of training and validation curves throughout all 15 epochs.

Compared to training from scratch, transfer learning reduced the number of trainable parameters from $\sim$138M to $\sim$26M, enabling faster convergence and better generalization on the $\sim$7,000-image training set.

VI. Conclusion and Future Work

This paper implemented VGGNet-16 with transfer learning in PyTorch for white blood cell classification on the BCCD dataset. A complete and reproducible pipeline was developed including data augmentation, ImageNet pretrained feature extraction, cosine LR scheduling, label smoothing, and early stopping. The model achieved:

- Test Accuracy: **91.63%**
- Macro F1-score: **0.9160**
- Weighted F1-score: **0.9158**
- Macro AUC-ROC: **0.9857**

Results demonstrate the viability of deep transfer learning for clinical blood cell analysis, with potential for deployment in automated hematology screening systems.

Future work can improve the system in several directions:

- Unfreeze deeper convolutional layers for end-to-end fine-tuning.
- Apply attention mechanisms or vision transformers (ViT).
- Use ensemble methods combining VGGNet-16 with ResNet or EfficientNet.
- Extend to larger or more fine-grained blood cell datasets.
- Add confidence-based rejection for low-certainty predictions.

Acknowledgment

The authors acknowledge Prof. (Dr.) Baij Nath Kaushik for guidance in the Deep Learning course and for framing this assignment on CNN implementation and evaluation. The model was trained using Google Colaboratory with NVIDIA T4 GPU resources.

References

[1] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2015.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[3] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, 2010.

[4] G. Litjens *et al.*, "A survey on deep learning in medical image analysis," *Med. Image Anal.*, vol. 42, pp. 60–88, 2017.

[5] D. Shen, G. Wu, and H.-I. Suk, "Deep learning in medical image analysis," *Annu. Rev. Biomed. Eng.*, vol. 19, pp. 221–248, 2017.

[6] R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi, "Convolutional neural networks: an overview and application in radiology," *Insights Imaging*, vol. 9, no. 4, pp. 611–629, 2018.

[7] C. Matek, S. Schwarz, K. Spiekermann, and C. Marr, "Human-level recognition of blast cells in acute myeloid leukaemia with convolutional neural networks," *Nat. Mach. Intell.*, vol. 1, no. 11, pp. 538–544, 2019.

[8] P. T. Mooney, "Blood Cell Images," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/paultimothymooney/blood-cells

[9] A. Acevedo *et al.*, "A dataset of microscopic peripheral blood cell images for development of automatic recognition systems," *Data Brief*, vol. 30, p. 105474, 2020.

[10] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, no. 1, p. 60, 2019.

[11] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.